

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М71-мм

Реализация OAuth 2.0 в основной системе электронных денег в соответствии со стандартами BI SNAP

Юсуп Амин Турмуди

Отчёт по преддипломной практике
в форме «Решение»

Научный руководитель:
доцент кафедры системного программирования, к. Ф.-м. н. доцент Д. В. Луцев

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	6
2. Обзор	7
2.1. Традиционные методы аутентификации API	7
2.2. JWT	8
3. Описание решения	11
3.1. Сравнение API Key, JWT и OAuth 2.0	12
4. Реализация системы	14
4.1. Архитектура приложения	14
4.2. Технологический стек и используемые библиотеки	15
4.3. Диаграмма вариантов использования	17
4.4. Процесс авторизации OAuth 2.0 (в соответствии с фрейм- ворком BI-SNAP).	18
4.5. Проектирование базы данных	29
4.6. Криптографические механизмы и формирование подписи	30
5. Тестирование и оценка	35
5.1. Функциональное тестирование	35
5.2. Тестирование производительности	40
Заключение	44
Список литературы	46

Введение

Развитие интернета — одно из самых значительных достижений человечества в 20 веке [1]. С помощью интернета люди могут мгновенно получать доступ к информации, знаниям, услугам и даже товарам из различных уголков мира [2]. Это удобство доступа стало движущей силой крупных изменений в бизнесе — перехода от традиционных (оффлайн) бизнес-моделей к цифровым (онлайн) бизнес-моделям, с основной целью охвата более широких рынков и достижения операционной эффективности.

Однако расширение рынка достигается не только через онлайн-сервисы, ориентированные непосредственно на конечных потребителей, но и через сотрудничество с другими компаниями в рамках схемы B2B¹. В экосистеме B2B компании взаимодействуют с целью предоставления услуг и обмена данными, что способствует укреплению их конкурентных позиций. Такое взаимодействие, как правило, реализуется посредством интеграции на основе API, обеспечивающей эффективное и масштабируемое межсистемное взаимодействие [3]. Это делает интеграцию систем приложений между компаниями необходимостью, которую невозможно избежать.

С увеличением сложности взаимодействий между компаниями вопрос безопасности становится первостепенным. В отличие от внутренних систем, которые включают только стороны внутри компании, системы B2B позволяют внешним сторонам получать доступ и даже управлять определенными данными. Поэтому системы B2B должны внедрять механизмы аутентификации (для проверки того, кто имеет доступ) и авторизации (для контроля того, что можно и нельзя использовать) [4].

В этом контексте недостаточно просто знать, кто получает доступ к системе. Сервер также должен понимать, какие действия разрешено выполнять этой стороне на ресурсном сервере. Цель — обеспечить, чтобы партнёры B2B могли получать доступ к данным только в рамках предоставленных прав доступа, не затрагивая чувствительную или нере-

¹Business to Business

levantную информацию. Некоторые широко используемые протоколы для защиты коммуникаций и доступа между приложениями включают OAuth 2.0 и SAML 2.0.² 2.0.

SAML 2.0 часто используется в традиционных корпоративных приложениях, таких как системы управления персоналом, бэк-офисные банковские системы и тому подобное [5]. В то время как OAuth 2.0 более часто используется для защиты HTTP-сервисов, особенно API³, которые являются основой обмена данными в современных приложениях.

В качестве примера, основная система приложения для электронных денег обычно реализует традиционные стандарты безопасности. В этой архитектуре каждый клиент, запрашивающий данные, включает API-ключ, который затем проверяется ресурсным сервером. Если API-ключ действителен, запрашиваемые данные обрабатываются и возвращаются клиенту. Хотя этот подход может быть еще допустим в рамках внутренней системы компании, риски безопасности увеличиваются, когда система электронных денег начинает интегрироваться с внешними сторонами в рамках B2B.

Таким образом, необходимо обновление системы безопасности, чтобы гарантировать защиту ресурсного сервера, даже если он открыт для сотрудничества с другими партнерами. Одним из решений, которое можно внедрить, является использование протокола OAuth 2.0.

Данная диссертационная работа посвящена исследованию и реализации механизмов аутентификации и авторизации на основе протокола OAuth 2.0, адаптированного в соответствии со стандартом BI-SNAP⁴. Предлагаемое решение реализовано на языке программирования Java с использованием фреймворка Spring. Соблюдение указанных регуляторных требований направлено на обеспечение безопасности ресурсного сервера в условиях B2B-взаимодействия, гарантируя защищённый доступ к данным, применение детализированного (fine-grained) управления

²Security Assertion Markup Language

³Application Programming Interface

⁴National Standard for Payments Open API, compiled and defined by Bank Indonesia

правами доступа, а также полное соответствие современным стандартам финансовой индустрии.

1. Постановка задачи

Основной целью данного исследования является реализация протокола OAuth 2.0 в соответствии со стандартом BI SNAP для обеспечения безопасности ресурсного сервера в контексте B2B-приложения. Применение OAuth 2.0 на основе стандарта BI SNAP направлено на формирование безопасного, стандартизированного и гибкого механизма управления процессами аутентификации и авторизации в соответствии с современными отраслевыми практиками. Для достижения поставленной цели были сформулированы следующие задачи исследования:

- Спроектировать и разработать сервер авторизации (Authorization Server) на основе протокола OAuth 2.0 с учетом требований стандарта BI SNAP с использованием фреймворка Spring. Данный сервер должен обеспечивать генерацию защищённых токенов доступа с управлением на основе областей доступа (scope) и ролей пользователей.
- Спроектировать и разработать API-шлюз в качестве централизованной точки входа для клиентских запросов, реализующий механизмы аутентификации и авторизации OAuth 2.0 в соответствии с требованиями стандарта BI SNAP, выполняющий валидацию запросов согласно положениям BI SNAP, а также обеспечивающий безопасное взаимодействие между клиентскими приложениями и сервером ресурсов.
- Интегрировать протокол OAuth 2.0 в клиентское приложение и ресурсный сервер, а также провести комплексное сквозное (end-to-end) тестирование для обеспечения соответствия механизмов аутентификации и авторизации спецификациям OAuth 2.0 и BI SNAP.
- Провести системную оценку безопасности и производительности реализованного решения, включая управление токенами, контроль доступа на основе scope, а также управление авторизацией для сторонних приложений в соответствии с требованиями BI SNAP.

2. Обзор

2.1. Традиционные методы аутентификации API

Взаимодействие между API внутри компании играет важную роль в интеграции систем и обмене данными между различными отделами. Основная цель такой коммуникации — формирование более полной и полезной информации, которая может поддерживать процессы принятия решений. Например, интеграция между бухгалтерией и отделом кадров позволяет компании автоматически и точно отображать информацию о зарплате сотрудника, его стаже и занимаемой должности.

В контексте внутренних систем компании аспект безопасности при обмене API обычно не отличается высокой сложностью. Поскольку все системы функционируют в контролируемой среде, традиционные методы аутентификации считаются достаточными. Одним из наиболее часто используемых методов аутентификации в таких случаях является API-ключ.

API-ключ — это уникальный токен, который клиент использует для доступа к API [6]. Обычно этот токен представляет собой случайную строку или определённый формат кодирования, например Base64. Механизм его работы достаточно прост: когда клиент хочет получить доступ к ресурсу сервера, он должен включить API-ключ в каждый запрос. Сервер затем проверяет API-ключ, чтобы убедиться, что запрос поступил от авторизованного источника. Ниже представлена базовая иллюстрация того, как работает API-ключ:

- Клиент отправляет запрос на определённую конечную точку, включая API-ключ в заголовок запроса или как параметр URL⁵.
- Ресурсный сервер проверяет действительность API-ключа.
- Если API-ключ действителен — предоставляется доступ к ресурсу; если нет — запрос отклоняется.

⁵Uniform Resource Locator

Хотя этот метод достаточно эффективен в ограниченных сценариях, таких как внутренняя среда компании, у него есть несколько существенных недостатков. API-ключи являются статичными и не распознают личность пользователя или уровень доступа. Кроме того, если API-ключ попадёт в руки злоумышленников, неавторизованные лица смогут легко получить доступ ко всем ресурсам без каких-либо ограничений.

2.2. JWT

JWT⁶— это открытый стандарт, определённый в RFC 7519, который предоставляет компактный и автономный механизм для безопасной передачи информации между сторонами в формате JSON-объекта [7]. JWT разработан таким образом, чтобы содержащаяся в нём информация могла быть проверена и заслуживала доверия, поскольку она подписана с использованием криптографических методов. Подпись может быть создана с помощью секретного ключа с алгоритмом HMAC⁷ или пары открытого и закрытого ключей с использованием алгоритмов RSA⁸ или ECDSA⁹ [8].

В последние годы использование JWT стало всё более популярным при разработке современных приложений, особенно веб- и мобильных. Такая популярность объясняется, в том числе, простотой использования протокола и высоким уровнем безопасности при передаче данных между компонентами системы. Кроме того, JWT не требует хранения состояния, что делает его особенно подходящим для использования в мобильных приложениях, распределённых архитектурах, таких как микросервисы, и одностраничных приложениях. [9].

Одним из практических примеров использования JWT является мобильное банковское приложение. Применение JWT в этом контексте основано на схожих принципах с его использованием в интернет-банкинге через веб-браузер, однако из-за различных характеристик и архитектуры

⁶JSON Web Token

⁷Hash-based Message Authentication Code

⁸Rivest–Shamir–Adleman

⁹Elliptic Curve Digital Signature Algorithm

мобильных приложений требуются некоторые важные адаптации.

Когда пользователь открывает мобильное банковское приложение и успешно входит в систему, используя действительные учетные данные — такие как имя пользователя и пароль или биометрическую аутентификацию отпечаток пальца или распознавание лица, — сервер генерирует JWT, содержащий информацию об аутентификации пользователя. Этот токен включает идентификатор пользователя, его роли или права доступа, а также срок действия токена. После успешного создания токен отправляется обратно в мобильное приложение в составе ответа сервера.

В отличие от браузеров, которые обычно хранят токены в cookies, мобильные приложения, как правило, хранят JWT в защищённом локальном хранилище, таком как Secure Storage или Keychain для iOS, а также EncryptedSharedPreferences или Android Keystore для Android. Эти механизмы хранения разработаны для предотвращения несанкционированного доступа, в том числе со стороны других приложений или при попытках эксплуатации на устройствах с root-доступом или jailbreak'ом.

Каждый раз, когда мобильное приложение отправляет запрос на сервер — например, чтобы проверить баланс счёта, просмотреть историю транзакций или выполнить перевод средств, — сохранённый JWT прикрепляется к заголовку Authorization с использованием схемы Bearer. Сервер получает запрос, проверяет подлинность токена с помощью зашифрованной подписи и удостоверяется, что срок действия токена ещё не истёк.

Если токен действителен и не просрочен, сервер обрабатывает запрос и возвращает соответствующий ответ. Однако если токен недействителен — например, если он был подделан, повреждён или срок его действия истёк, — сервер отклонит запрос и потребует от приложения повторной аутентификации пользователя.

Несмотря на то что JWT весьма эффективно используется для аутентификации конечных пользователей, в более масштабных сценариях, таких как интеграция систем B2B, применение OAuth 2.0 в качестве рамочной системы авторизации значительно важнее и настоятельно

рекомендуется.

3. Описание решения

Для защиты и авторизации доступа к серверу ресурсов во внутренней среде — всё ещё в рамках одной компании — использование API-ключей и JSON Web Tokens (JWT) обычно является достаточным. Оба механизма обеспечивают базовый контроль над тем, кто может получить доступ к API, и позволяют эффективно проверять личность клиента. JWT, в частности, предоставляет преимущества за счёт возможности включать утверждения (claims) и задавать срок действия токена, что делает его удобным для множества внутренних задач авторизации.

Однако, когда система развивается и начинает взаимодействовать с внешними системами — будь то между отделами, дочерними компаниями или даже сторонними организациями — одних только API-ключей и JWT уже недостаточно. Основная причина в трудностях управления авторизацией, отзывом токенов, более детализированным контролем доступа и необходимости получения явного согласия от владельца ресурса.

Именно здесь OAuth 2.0 становится более комплексным решением. OAuth 2.0 — это фреймворк авторизации, который позволяет безопасно получать доступ к ресурсам без прямой передачи учётных данных пользователя [10]. С помощью OAuth владелец ресурса (обычно пользователь) может предоставить ограниченные права доступа клиенту (стороннему приложению) через механизм токенов авторизации. Это особенно полезно в системах с участием нескольких организаций, где требуется гибкое и стандартизированное управление доступом.

Кроме того, OAuth 2.0 поддерживает различные потоки авторизации, такие как: Authorization Code Flow, Client Credentials Flow, Implicit Flow. Эти потоки можно адаптировать под конкретные сценарии — будь то веб-приложения, мобильные приложения или взаимодействие сервер-сервер. Всё это делает OAuth 2.0 рекомендуемым стандартом для интеграции между системами, где требуется высокий уровень безопасности, детализированная авторизация и масштабируемость в долгосрочной перспективе.

В заключение, для простых задач внутренней интеграции API-ключи и JWT остаются актуальными и эффективными. Однако для более сложных сценариев авторизации с участием нескольких сторон внедрение OAuth 2.0 — это стратегический шаг, соответствующий современным лучшим практикам в области безопасности.

3.1. Сравнение API Key, JWT и OAuth 2.0

Для более чёткого понимания эволюции механизмов аутентификации и авторизации в Таблице 1 представлено сравнительное исследование API Key, JSON Web Token (JWT) и OAuth 2.0. Сравнение выполнено на основе ключевых характеристик безопасности и архитектуры, включая идентификацию субъекта доступа, управление жизненным циклом токенов, уровень детализации контроля доступа и применимость в сценариях взаимодействия типа B2B.

Таблица 1: Сравнение API Key, JWT и OAuth 2.0

Характеристика	API Key	JWT	OAuth 2.0
Идентификация субъекта	Нет	Частично	Да
Контроль срока действия	Нет	Да	Да
Отзыв токена	Нет	Ограничено	Да
Гранулярный контроль доступа	Нет	Ограничено	Да
Применимость в B2B	Нет	Средняя	Высокая

Как показано в Таблице 1, API Key представляет собой базовый механизм аутентификации, который не обеспечивает таких важных возможностей, как контроль срока действия и детализированное управление доступом. Использование JWT улучшает данный подход за счёт внедрения безсессионной аутентификации с включением утверждений (claims) и ограниченного срока действия токена, однако данный механизм не предоставляет стандартизированного способа отзыва токенов или делегированной авторизации.

В отличие от этого, OAuth 2.0 представляет собой комплексную и гибкую модель авторизации, обеспечивающую делегирование доступа, управление жизненным циклом токенов и контроль прав доступа

на основе областей (scopes). Эти характеристики делают OAuth 2.0 наиболее подходящим решением для сложных и масштабируемых B2B-систем, где требуется безопасное взаимодействие между независимыми участниками.

4. Реализация системы

4.1. Архитектура приложения

Архитектура, применяемая при разработке системы безопасности на основе протокола OAuth 2.0, реализована с использованием подхода микросервисов. В рамках данной архитектуры процессы аутентификации и авторизации как клиентов, так и пользователей централизованы в выделенном сервисе, известном как сервер авторизации. В дальнейшем проверка валидности токена доступа осуществляется отдельным компонентом — API-шлюзом — перед тем как запросы направляются к соответствующим сервисам. Подобное разделение ответственности обеспечивает четкую декомпозицию функций системы, что способствует повышению уровня безопасности, масштабируемости и сопровождаемости.

Выбор микросервисной архитектуры обусловлен необходимостью обеспечения гибкости и расширяемости системы в перспективе. Данный подход позволяет реализовывать новые функциональные возможности, такие как регистрация клиентов, регистрация карт, подтверждение операций с помощью одноразовых паролей (ОТР), настройка лимитов по картам и реализация платежных сервисов, в модульной форме без нарушения работы существующих компонентов. Кроме того, каждый сервис может разрабатываться, тестироваться и развертываться независимо, что ускоряет процесс разработки и снижает риски отказа системы в целом.

Дополнительно, использование API-шлюза в качестве единой точки входа обеспечивает централизованную реализацию механизмов безопасности, включая ограничение частоты запросов (rate limiting), валидацию входящих данных, ведение журналов (logging) и мониторинг. API-шлюз также гарантирует, что каждый входящий запрос содержит корректный и действительный токен доступа перед его передачей в целевые сервисы. Это позволяет снизить нагрузку на отдельные сервисы в части проверки безопасности и обеспечить единообразное применение политик

безопасности.

Более того, разделение сервера авторизации и ресурсных серверов создает условия для использования протокола OAuth 2.0 и его продвинутых механизмов защиты, таких как использование refresh-токенов, управление сроком действия токенов и механизмы их отзыва (revocation). Эти меры играют ключевую роль в снижении рисков несанкционированного использования токенов и повышении контроля над пользовательскими сессиями. Данная архитектура также обеспечивает возможность интеграции с внешними системами, включая банковские сервисы и платежные шлюзы, при сохранении конфиденциальности и целостности пользовательских данных и транзакций.

4.2. Технологический стек и используемые библиотеки

Система реализована с использованием языка программирования Java и построена на основе экосистемы Spring. Реализация системы соответствует архитектуре, описанной в предыдущем разделе; соответственно, каждый компонент разрабатывается с учетом требований масштабируемости, сопровождаемости и безопасности.

Ключевым компонентом уровня безопасности является сервер авторизации, реализованный с использованием Spring Authorization Server [11], построенного поверх Spring Security [12]. Данный компонент отвечает за процессы аутентификации и выдачу токенов доступа в соответствии со спецификациями OAuth 2.1 и OpenID Connect 1.0. По сравнению с коммерческими или сторонними OAuth-решениями данный подход обеспечивает полный контроль над механизмами безопасности и возможностями кастомизации, включая использование симметричных и асимметричных криптографических алгоритмов.

Генерация и валидация токенов выполняются с использованием JSON Web Token (JWT). Применение JWT позволяет реализовать модель stateless-аутентификации, исключая необходимость хранения сессий на стороне сервера, что повышает масштабируемость системы. Spring

Security обеспечивает базовую инфраструктуру для защиты конечных точек и реализации политик контроля доступа.

Для хранения данных сервер авторизации использует Spring Data JPA в сочетании с реляционной базой данных PostgreSQL [13]. Данный уровень отвечает за хранение пользовательских данных, информации о клиентских приложениях и данных, связанных с авторизацией. Такой подход обеспечивает надежность и согласованность критически важных данных.

Помимо сервера авторизации, система включает API Gateway, реализованный с использованием Spring Cloud Gateway на базе реактивного фреймворка WebFlux [14]. Gateway выполняет функцию единой точки входа для всех клиентских запросов и отвечает за их маршрутизацию к соответствующим микросервисам.

Использование WebFlux обеспечивает неблокирующую асинхронную обработку запросов, что повышает производительность системы при высокой нагрузке и позволяет эффективно обрабатывать большое количество одновременных соединений [15]. На уровне gateway обеспечивается проверка JWT-токенов, содержащихся в запросах, что гарантирует передачу только аутентифицированных и авторизованных запросов внутренним сервисам.

Для поддержки реактивных операций gateway использует R2DBC для неблокирующего взаимодействия с базой данных PostgreSQL [16]. Это может применяться для вспомогательных задач, таких как логирование, аудит или проверка токенов. Кроме того, используются пользовательские фильтры gateway для обработки сквозных задач, включая аутентификацию, валидацию запросов, логирование и трансформацию данных.

Выбор технологий, включая Spring Authorization Server и Spring Cloud Gateway, обусловлен их способностью обеспечивать комплексную поддержку реализации безопасности на основе OAuth 2.1. Spring Authorization Server обеспечивает управление аутентификацией и выдачу токенов в соответствии с отраслевыми стандартами, тогда как Spring Cloud Gateway предоставляет гибкие механизмы маршрутизации

и фильтрации для защиты входящих запросов. В совокупности использование Spring Security, JWT, а также реактивного подхода на основе WebFlux и R2DBC обеспечивает создание безопасной, эффективной и масштабируемой системы. В целом данная комбинация технологий способствует более структурированному, согласованному и расширяемому процессу разработки, соответствующему требованиям системы.

4.3. Диаграмма вариантов использования

После определения архитектуры системы и описания выбора используемых библиотек следующим этапом является анализ взаимодействия между компонентами системы. Данный этап имеет особую значимость в контексте интеграции на основе OAuth с сервисами электронных денежных средств (e-money). На начальном этапе приложение взаимодействует с сервером авторизации для получения B2B-токена, который выполняет функцию основного идентификатора на уровне системы. Данный токен представляет само приложение и используется для установления доверия между системами. Для операций, требующих учета конкретного пользовательского контекста, приложение запрашивает B2B2C-токен. Данный токен напрямую связан с учетной записью пользователя и содержит права доступа, необходимые для выполнения операций на уровне пользователя. После выдачи токен включается в каждый запрос, отправляемый через API Gateway. Шлюз проверяет валидность токена, и в случае успешной проверки запрос перенаправляется к внутренним сервисам или сервису электронных денежных средств. Выполняемые операции могут включать проверку баланса или перевод средств. Чтобы прояснить вышеизложенное, ниже представлена диаграмма вариантов использования, иллюстрирующая поток системы.

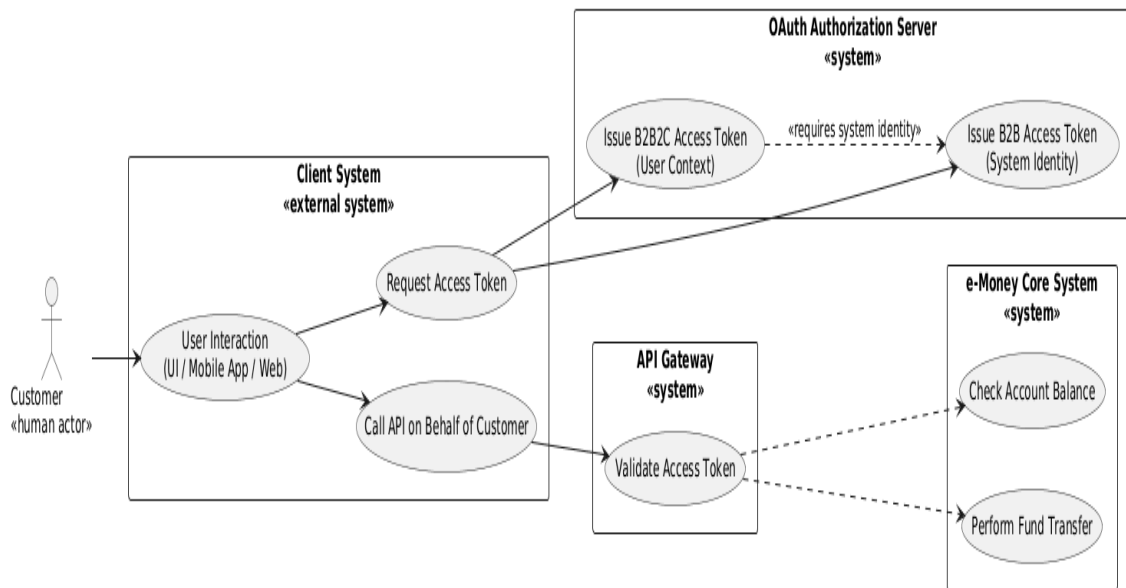


Рис. 1: Диаграмма вариантов использования сервера OAuth

4.4. Процесс авторизации OAuth 2.0 (в соответствии с фреймворком BI-SNAP).

BI-SNAP¹⁰ — это национальный стандарт, установленный Банком Индонезии с целью регулирования взаимодействия между поставщиками услуг платёжных систем [17]. Этот стандарт применяется не только к банковским учреждениям, но также является обязательным для небанковских организаций, таких как PSP¹¹, включая финтех-компании и стартапы, работающие в экосистеме цифровых платежей Индонезии.

Одним из ключевых компонентов BI-SNAP является внедрение протокола безопасности OAuth 2.0. OAuth 2.0 используется как стандарт авторизации, обеспечивающий безопасный, аутентифицированный доступ между системами, соответствующий правам доступа, предоставленным пользователем. Цель использования протокола OAuth 2.0 в рамках BI-SNAP — обеспечение конфиденциальности данных клиентов, предотвращение несанкционированного доступа, а также упрощение интеграции между участниками платёжной индустрии.

¹⁰National Standard for Open API Payments

¹¹Payment Service Providers

Для реализации OAuth 2.0 в рамках BI-SNAP используются различные механизмы авторизации в зависимости от модели взаимодействия между системами и пользователями. В целом, OAuth 2.0 определяет несколько типов grant, каждый из которых предназначен для конкретных сценариев использования и требований безопасности.

В контексте данного исследования рассматриваются два основных потока: Client Credentials Flow для взаимодействия между системами (B2B) и Authorization Code Flow для сценариев с участием конечного пользователя (B2B2C). Каждый из этих потоков определяет процесс получения токена доступа и его последующего использования для доступа к защищённым ресурсам.

В следующих подразделах подробно описываются оба потока, включая процесс получения токена доступа и механизм доступа к серверу ресурсов с использованием выданного токена.

4.4.1. Поток Client Credentials и доступ к ресурсам в B2B-взаимодействии

Данный поток применяется для взаимодействия между информационными системами без участия конечного пользователя. Он охватывает как процесс получения токена доступа с использованием механизма Client Credentials Flow, так и его последующее применение при обращении к защищённым ресурсам.

Поток Client Credentials.

В механизме аутентификации клиента с использованием протокола OAuth 2.0 по схеме client credentials процесс получения токена доступа начинается с отправки клиентом запроса на сервер авторизации (Authorization Server). Запрос включает заголовки и тело, содержащие обязательные и опциональные параметры в соответствии с установленной спецификацией.

После получения запроса сервер авторизации выполняет последовательность процедур валидации, направленных на проверку наличия всех обязательных параметров, регистрации и авторизации клиента, а также корректности цифровой подписи, обеспечивающей целостность

и подлинность запроса. Данный этап гарантирует, что к дальнейшей обработке допускаются только легитимные клиенты.

В случае успешного прохождения всех этапов проверки сервер авторизации формирует токен доступа, который отражает идентичность клиента и его права доступа к серверу ресурсов. Сформированный токен возвращается клиенту и используется в качестве учетных данных для аутентификации. На рисунке ниже представлена диаграмма последовательностей, иллюстрирующая процесс получения токена доступа.

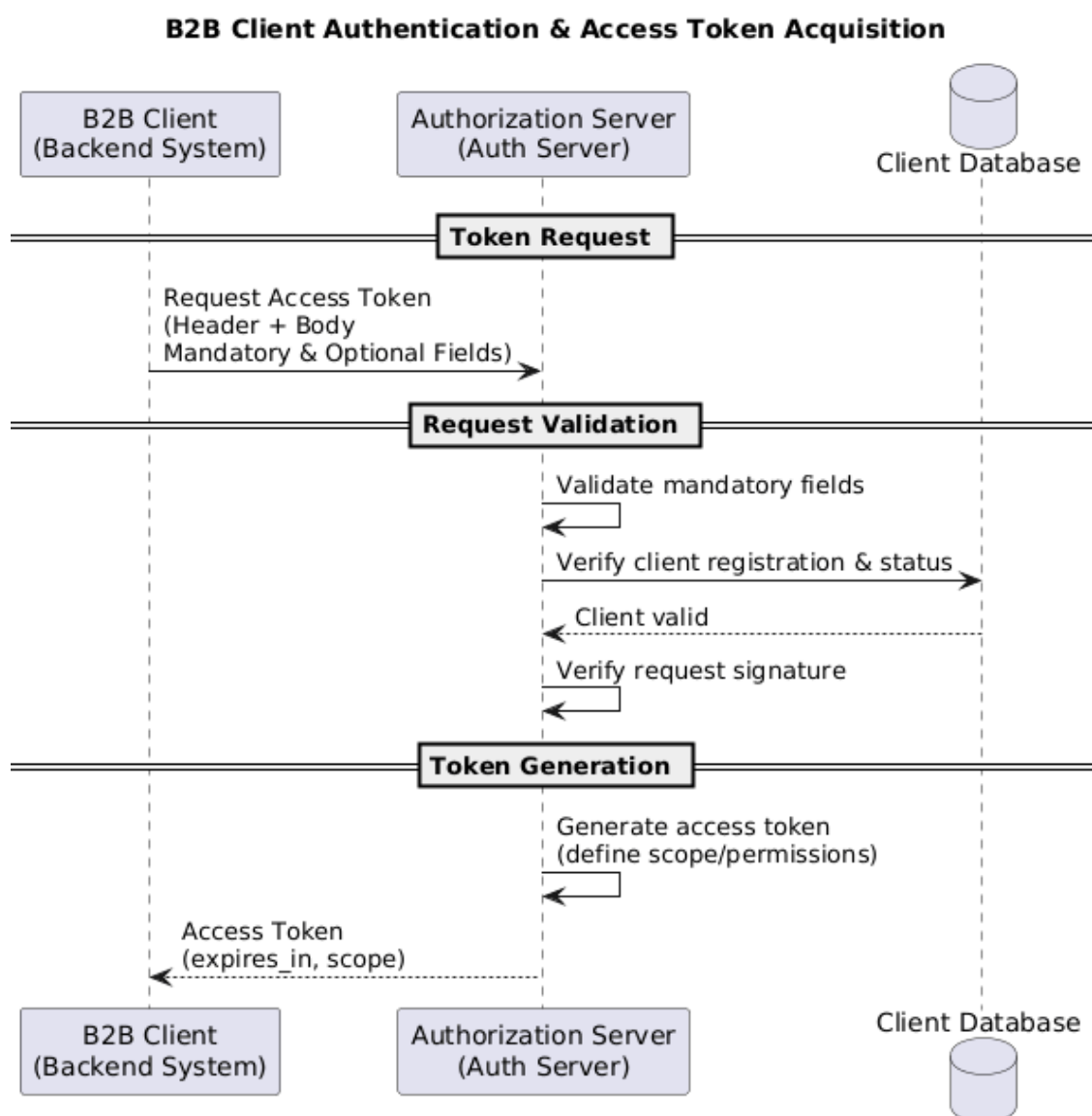


Рис. 2: Токен доступа B2B

Доступ к серверу ресурсов с использованием токена доступа.

После завершения процесса Client Credentials Flow клиент получает токен доступа, который в дальнейшем используется в качестве учетных данных для обращения к серверу ресурсов (Resource Server). При каждом запросе к серверу ресурсов клиент обязан передавать заголовки и тело запроса, содержащие обязательные параметры в соответствии с установленной спецификацией. Все входящие запросы первоначально обрабатываются API-шлюзом, выступающим в качестве единой точки входа в систему. На данном этапе выполняется последовательность процедур валидации, включающая проверку наличия обязательных параметров в заголовках и теле запроса, верификацию цифровой подписи для обеспечения целостности и подлинности запроса, а также проверку токена доступа, включая контроль срока его действия. В случае успешного прохождения всех этапов валидации API-шлюз перенаправляет запрос на сервер ресурсов для дальнейшей обработки. После завершения обработки сервер ресурсов возвращает ответ API-шлюзу, который, в свою очередь, передает его обратно клиенту. Данный механизм обеспечивает безопасный, контролируемый и соответствующий установленным политикам безопасности доступ к защищённым ресурсам. На рисунке ниже представлена диаграмма последовательностей, иллюстрирующая процесс выполнения запроса к серверу ресурсов.

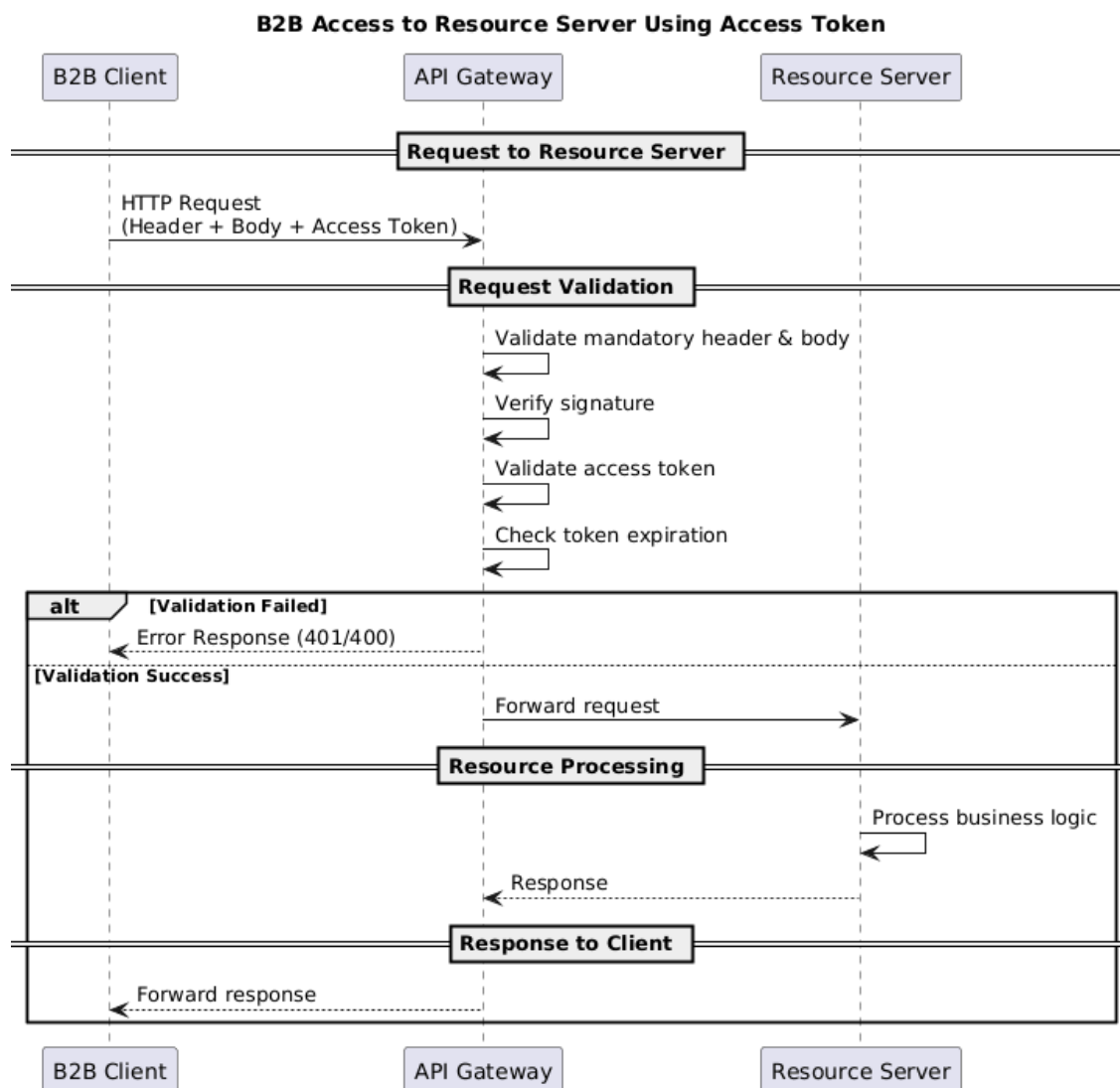


Рис. 3: Процесс обращения клиента к серверу ресурсов через API-шлюз с использованием access token в архитектуре B2B

4.4.2. Авторизация B2B2C с использованием потока Authorization Code

Данный поток применяется в сценариях, предполагающих участие конечного пользователя, и обеспечивает повышенный уровень безопасности за счёт многоэтапного процесса авторизации и передачи данных.

Запрос авторизации и получение authorization code.

Начальным этапом, предшествующим доступу конечного пользователя к resource server в архитектуре OAuth 2.0, является процесс authorization request. Данный процесс инициируется посредством взаимодействия

пользователя с client application, как правило, через элемент пользовательского интерфейса, такой как кнопка login или authorize. Этот элемент выступает в качестве точки входа (entry point), запускающей процесс авторизации.

При активации данного элемента client application формирует и отправляет authorization request на authorization server через authorization endpoint. Запрос, как правило, включает ключевые параметры, такие как client identifier, redirect URI, response type и scope, определяющие уровень запрашиваемого доступа.

После получения запроса authorization server выполняет процедуру валидации, включая проверку того, что client зарегистрирован и имеет право использовать сервис авторизации. Далее система проверяет статус аутентификации пользователя. Если пользователь не аутентифицирован, он перенаправляется на страницу login для ввода корректных учетных данных.

После успешной аутентификации authorization server отображает страницу consent, на которой пользователю предоставляется информация о запрашиваемых правах доступа (scope) со стороны client application. На данном этапе пользователь обладает полным контролем и может либо подтвердить, либо отклонить запрос.

В случае предоставления согласия authorization server генерирует authorization code и передает его обратно в client application через заранее зарегистрированный redirect URI. Данный authorization code является результатом этапа authorization request и используется на последующих этапах процесса OAuth 2.0.

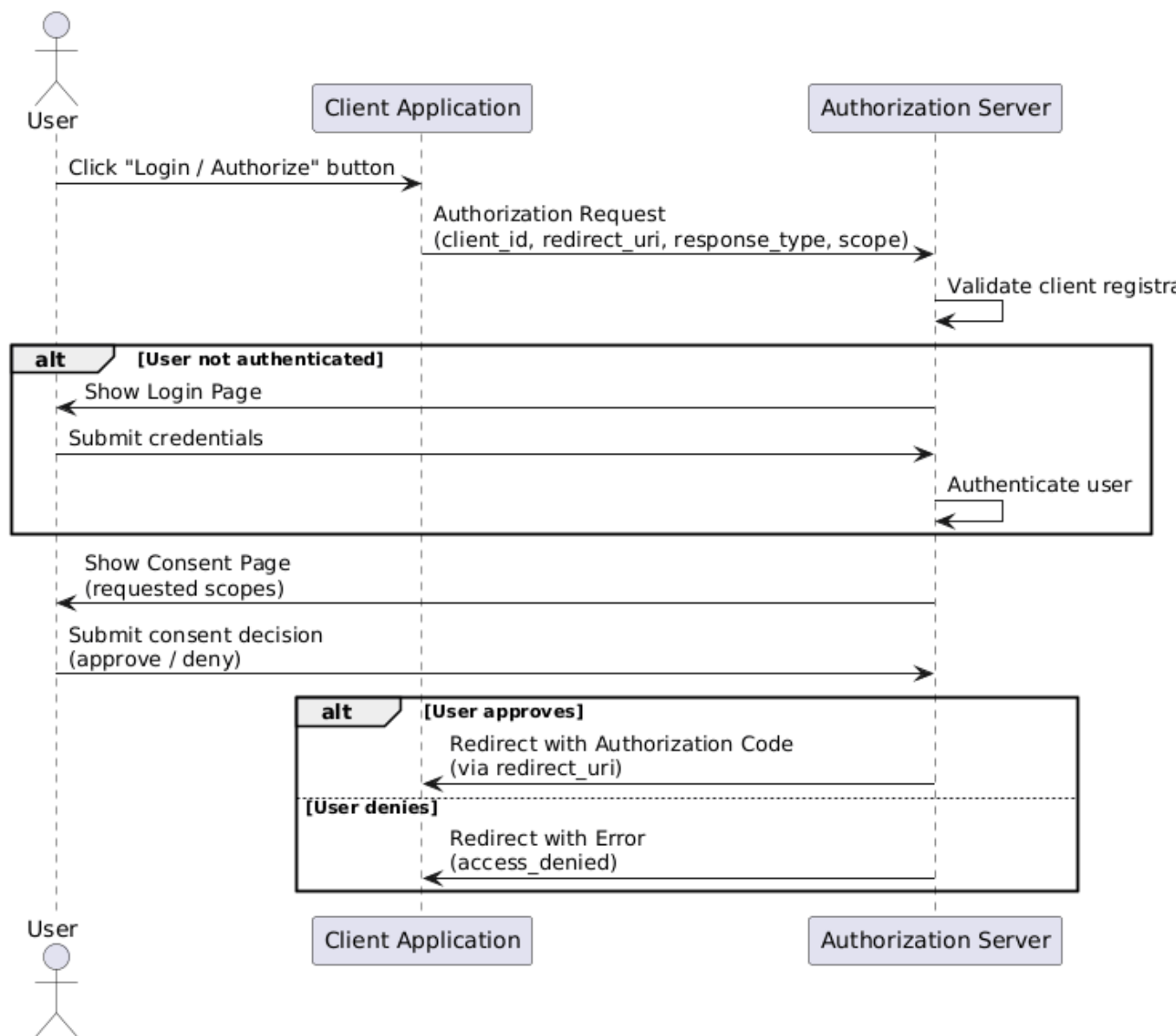


Рис. 4: Процесс запроса авторизации в OAuth 2.0

Обмен authorization code на токен доступа.

После получения authorization code приложением client application следующим этапом является процесс запроса access token в контексте архитектуры B2B2C. Перед отправкой запроса client обязан включить все обязательные параметры в body запроса, тогда как элементы безопасности, такие как signature, передаются в header в соответствии со спецификацией authorization server.

После получения запроса authorization server сначала выполняет первичную проверку структуры запроса, включая наличие header и

body. Далее осуществляется проверка signature, содержащейся в header, с целью обеспечения целостности и подлинности запроса. Данный этап является критически важным, поскольку данные, содержащиеся в body, включая authorization code, могут считаться достоверными только после подтверждения подлинности запроса.

После успешной проверки signature authorization server переходит к проверке подлинности client, удостовераясь, что он зарегистрирован и имеет право выполнять обмен токена. Затем выполняется проверка authorization code, включая подтверждение того, что код не истёк (not expired), не был использован ранее и соответствует запрашивающему client.

Если все этапы проверки успешно пройдены, authorization server генерирует и возвращает access token приложению client application.

Access token представляет собой делегированные права доступа, предоставленные client application для выполнения действий от имени конечного пользователя (end user). В архитектуре B2B2C данный токен содержит информацию об идентичности пользователя и утверждённой области доступа (scope), что позволяет client получать доступ к защищённым ресурсам на resource server в соответствии с предоставленными правами.

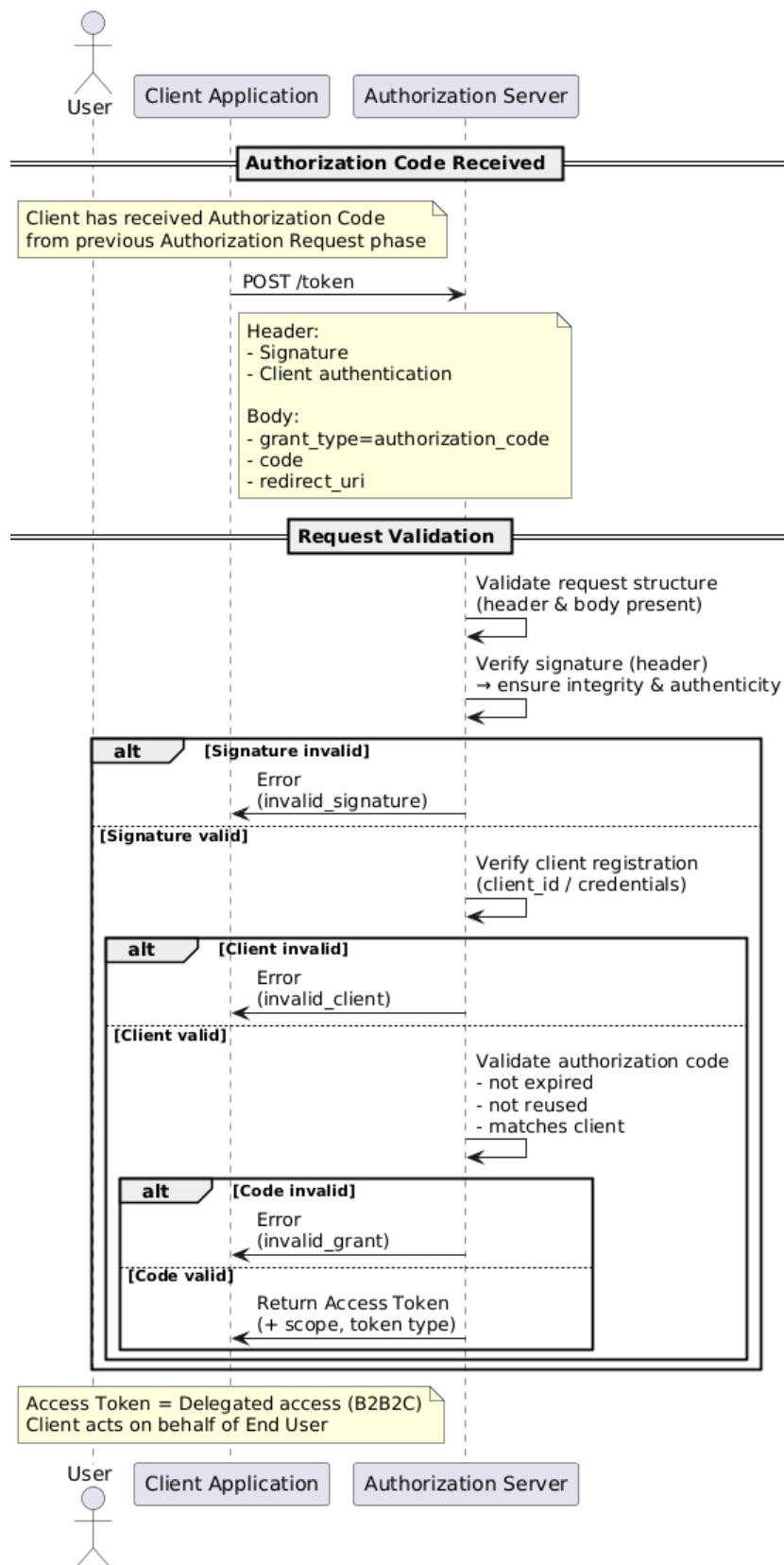


Рис. 5: Процесс обмена authorization code на access token в OAuth 2.0 в архитектуре B2B2C

Доступ к ресурсам от имени пользователя.

На заключительном этапе client application подготавливает все обязательные параметры в header и body запроса, включая access token для B2B-сервисов, access token для B2B2C-сервисов, а также signature в качестве механизма обеспечения безопасности.

При получении запроса API gateway выступает в качестве основной точки входа и выполняет процедуру валидации header и body. Данный процесс включает проверку полноты обязательных параметров, валидацию access token, а также проверку signature для обеспечения целостности и подлинности запроса.

В случае успешного прохождения всех этапов валидации API gateway перенаправляет запрос на resource server для дальнейшей обработки. После завершения обработки resource server возвращает ответ в API gateway, который, в свою очередь, передает его обратно в client application.

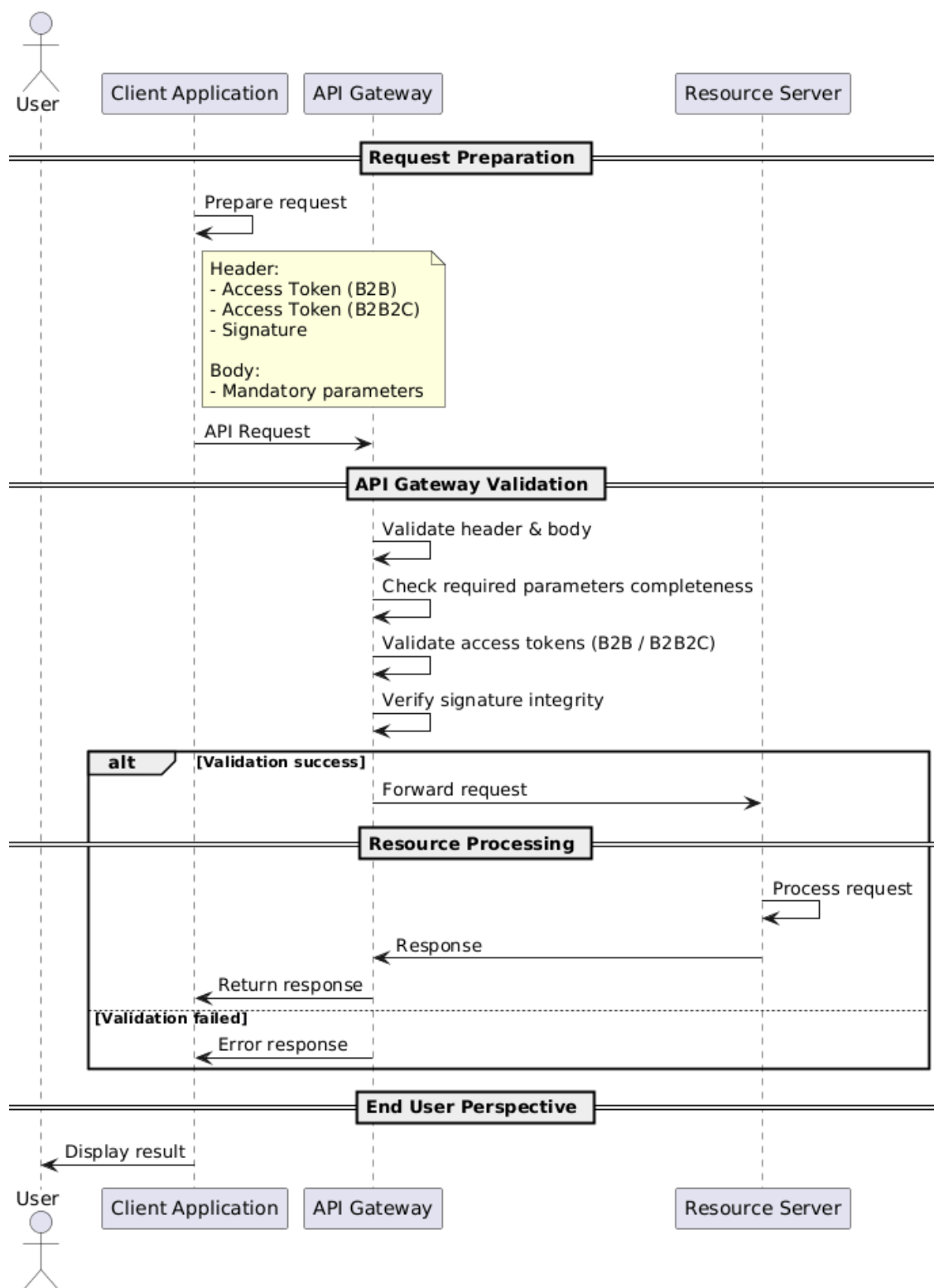


Рис. 6: Процесс доступа к серверу ресурсов через API-шлюз с использованием access token B2B и B2B2C

4.5. Проектирование базы данных

Для соблюдения положений и требований BI-SNAP необходимо разработать модель базы данных, которая сможет поддерживать процесс аутентификации и авторизация. Данная модель базы данных используется для хранения списка клиентов вместе с их правами доступа, с которыми было установлено партнёрское сотрудничество.

Благодаря данному механизму система может предотвращать несанкционированный доступ со стороны клиентов, которые не имеют партнёрских отношений или не выполнили требования интеграции с Core System электронных денег (E-money). Проверка выполняется на основе данных, хранящихся в базе данных, что позволяет обеспечить целостность и безопасность системы. Для большей наглядности ниже представлена модель базы данных.

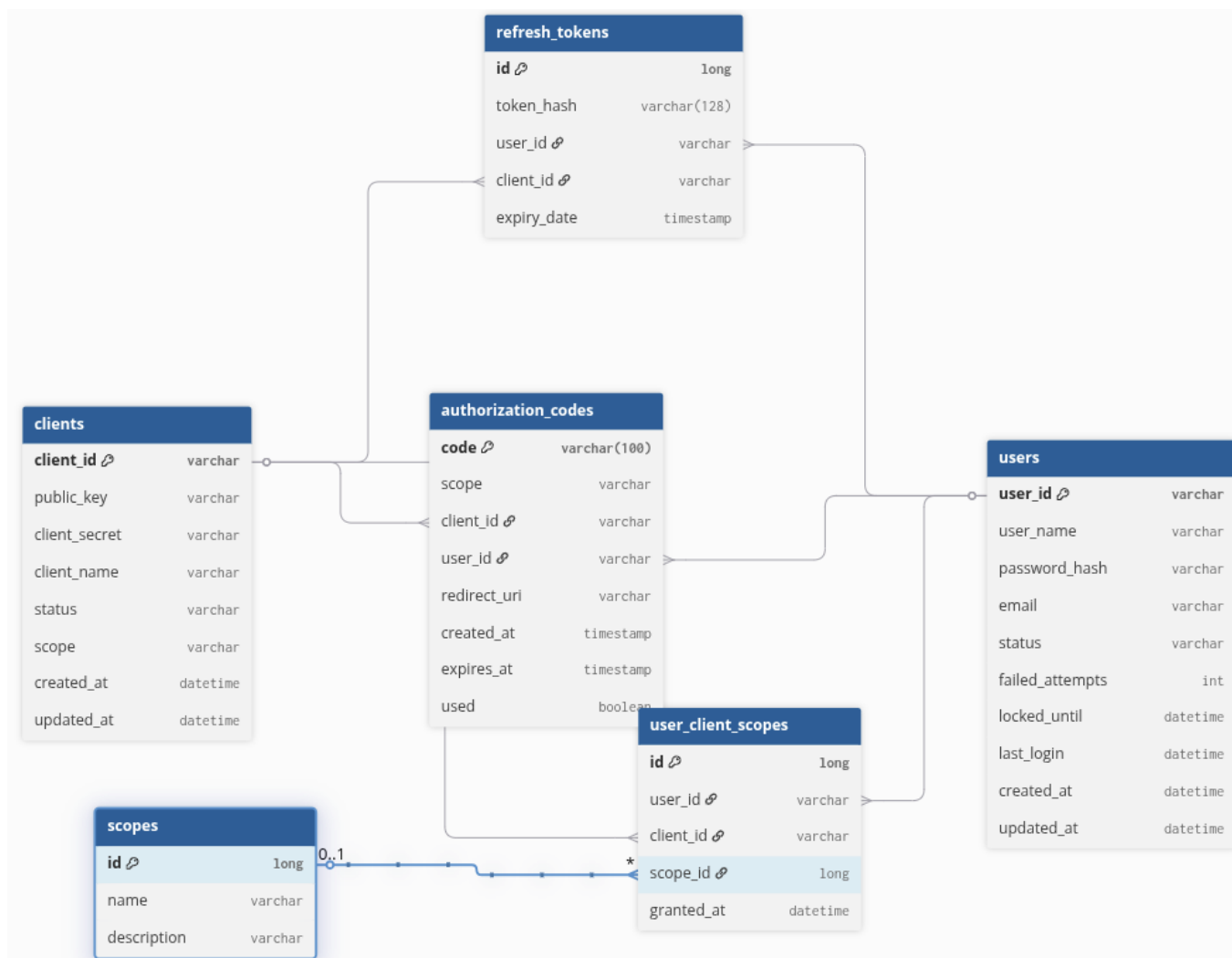


Рис. 7: Проектирование базы данных BI SNAP

4.6. Криптографические механизмы и формирование подписи

В продолжение предыдущего раздела в данной части представлено детальное рассмотрение механизма валидации клиента сервером авторизации с использованием цифровой подписи, сформированной на основе асимметричной криптографии. Асимметричная криптография представляет собой класс криптографических систем, основанных на использовании пары математически взаимосвязанных ключей: открытого ключа (public key) и закрытого ключа (private key). В рамках взаимодействия клиента с OAuth-сервером клиент обязан сгенерировать и поддерживать такую пару ключей как часть доверенной инфраструктуры. Закрытый

ключ хранится в строгой конфиденциальности у клиента и используется исключительно для формирования цифровой подписи. В свою очередь, открытый ключ передаётся и регистрируется на стороне OAuth-сервера, где применяется для проверки подлинности подписи. При формировании запроса на получение токена доступа — как в сценариях B2B (Business-to-Business), так и B2B2C (Business-to-Business-to-Consumer) — клиент подписывает полезную нагрузку запроса с использованием своего закрытого ключа. После получения запроса сервер авторизации использует соответствующий зарегистрированный открытый ключ для выполнения следующих проверок:

- Аутентификация: подтверждение того, что запрос исходит от легитимного и ранее зарегистрированного клиента.
- Проверка целостности: обеспечение того, что переданные данные не были изменены в процессе передачи.

Данный механизм гарантирует, что только субъект, обладающий корректным закрытым ключом, способен сформировать подпись, которая может быть успешно проверена с использованием соответствующего открытого ключа. Таким образом, обеспечиваются высокие гарантии подлинности и целостности данных в процессе взаимодействия. В следующем разделе представлены детали реализации, включая примеры кода для генерации и проверки цифровой подписи.

```
1 public static String createSignature(String data, PrivateKey privateKey)
   throws Exception {
2     Signature signer = Signature.getInstance("SHA256withRSA");
3     signer.initSign(privateKey);
4     signer.update(data.getBytes(StandardCharsets.UTF_8));
5
6     byte[] signatureBytes = signer.sign();
7     return Base64.getEncoder().encodeToString(signatureBytes);
8 }
```

Листинг 1: Generates a Base64-encoded digital signature using SHA256withRSA

```

1 public static boolean verifySignature(String data, String signatureStr,
2     PublicKey publicKey)
3     throws Exception {
4
5     Signature verifier = Signature.getInstance("SHA256withRSA");
6     verifier.initVerify(publicKey);
7     verifier.update(data.getBytes(StandardCharsets.UTF_8));
8
9     byte[] signatureBytes = Base64.getDecoder().decode(signatureStr);
10    return verifier.verify(signatureBytes);
11 }

```

Листинг 2: Verifies a digital signature using the corresponding public key

На основе представленной реализации кода, строка данных, используемая в качестве входных данных как для процесса генерации цифровой подписи, так и для процесса её проверки, формируется путём конкатенации параметров *ClientID* и *X_TIMESTAMP*, разделённых символом вертикальной черты (|).

Формально данная конструкция может быть представлена следующим образом:

`ClientID + "|" + X_TIMESTAMP`

С точки зрения криптографии, данная строка представляет собой сообщение, обрабатываемое с использованием схемы цифровой подписи с открытым ключом, а именно комбинации криптографической хеш-функции SHA-256 и алгоритма RSA (*SHA256withRSA*) [18, 19]. В процессе формирования подписи сообщение сначала обрабатывается с помощью хеш-функции SHA-256, что позволяет получить хеш-значение фиксированной длины, обладающее свойствами стойкости к коллизиям и необратимости [20].

Далее полученное хеш-значение шифруется с использованием закрытого ключа в рамках алгоритма RSA, формируя цифровую подпись [21]. В процессе проверки выполняется повторное вычисление хеша исходного сообщения и его сравнение с результатом расшифрования

подписи с использованием открытого ключа. Совпадение этих значений подтверждает целостность и подлинность сообщения [22].

Таким образом, использование комбинированной строки *ClientID* и *X_TIMESTAMP* обеспечивает уникальность сообщения и поддерживает ключевые свойства безопасности, такие как целостность, аутентичность и неотказуемость, а также обеспечивает защиту от атак повторного воспроизведения (*replay attack*).

В дополнение к механизму асимметричной криптографии, применяемому при получении токена доступа, система также использует симметричную криптографию при обращении к защищённым ресурсам. На данном этапе используется общий секрет (*client secret*) для формирования и проверки подписи запроса с применением алгоритма HMAC-SHA512, являющегося стандартизированным механизмом аутентификации сообщений на основе хеш-функций [23, 24, 25]. Сформированная подпись передаётся в HTTP-заголовке *X-Signature*.

Подпись формируется на основе строки *stringToSign*, определяемой следующим образом:

```
HTTPMethod + ":" + EndpointUrl + ":" + AccessToken + ":" +  
Lowercase(HexEncode(SHA-256(minify(RequestBody)))) + ":" +  
TimeStamp
```

Компонент *RequestBody* предварительно подвергается минимизации (*minification*) с целью удаления несущественных элементов JSON-структуры [26]. Затем к результату применяется хеш-функция SHA-256 [20], после чего выполняется преобразование в шестнадцатеричный формат и нормализация к нижнему регистру для обеспечения единообразия представления. В случае отсутствия тела запроса используется пустая строка.

Далее строка *stringToSign* обрабатывается алгоритмом HMAC-SHA512 с использованием *client secret* в качестве криптографического ключа. На стороне сервера проверка осуществляется путём повторного формирования *stringToSign* и сравнения вычисленной подписи с полученным значением. Данный механизм обеспечивает целостность и

аутентичность сообщения, а также снижает риск атак повторного воспроизведения за счёт использования временной метки [27, 28]. Реализация формирования и проверки подписи на основе HMAC-SHA512 представлена в следующих листингах, что позволяет обеспечить наглядность и полноту описания предложенного механизма безопасности.

```
1 public static String sign(String message, SecretKey key) throws Exception {  
2     Mac mac = Mac.getInstance("HmacSHA512");  
3     mac.init(key);  
4  
5     byte[] rawHmac = mac.doFinal(message.getBytes(StandardCharsets.UTF_8));  
6     return Base64.getEncoder().encodeToString(rawHmac);  
7 }
```

Листинг 3: Generates a Base64-encoded HMAC-SHA512 signature using a shared secret key

```
1 public static boolean verify(String message, String signature, SecretKey key)  
    throws Exception {  
2     String computed = sign(message, key);  
3  
4     return MessageDigest.isEqual(  
5         computed.getBytes(StandardCharsets.UTF_8),  
6         signature.getBytes(StandardCharsets.UTF_8)  
7     );  
8 }
```

Листинг 4: Verifies an HMAC-SHA512 signature using constant-time comparison

5. Тестирование и оценка

5.1. Функциональное тестирование

Функциональное тестирование проводится с целью обеспечения соответствия всех функций приложения установленным требованиям. Разработанная система реализует протокол OAuth 2.0 в соответствии со стандартами, установленными Bank Indonesia через спецификацию BI SNAP. В связи с этим все процессы системы должны соответствовать требованиям, определённым в данных стандартах, особенно в части механизмов аутентификации и авторизации. В процессе тестирования каждая функция проверяется на основе заранее разработанных сценариев, охватывающих основные этапы работы системы, включая получение B2B access token, авторизацию пользователя, процесс входа, управление согласием (consent), выдачу B2B2C access token, а также доступ к ресурсному серверу. Целью тестирования является подтверждение того, что каждый этап протокола OAuth 2.0 функционирует корректно и формирует ответы, соответствующие установленным требованиям. Кроме того, функциональное тестирование направлено на выявление возможных ошибок или отклонений в реализации системы относительно стандарта BI SNAP. Проведение данного тестирования позволяет обеспечить высокий уровень надёжности, безопасности и интероперабельности системы в соответствии с требованиями, установленными Bank Indonesia.

5.1.1. Тестовые случаи

Функциональное тестирование проводится путём разработки набора тест-кейсов, отражающих основные этапы работы системы, включая получение B2B access token, авторизацию и аутентификацию пользователя, управление согласием (consent), выдачу B2B2C access token, а также доступ к ресурсному серверу. Разработка тест-кейсов основывается на реализации протокола OAuth 2.0, а также стандарта BI SNAP, установленного Bank Indonesia. Каждый тест-кейс предназначен для проверки соответствия между входными данными и результатами, генерируемыми

системой, как в нормальных условиях, так и в условиях возникновения ошибок. Результаты тестирования используются для подтверждения того, что все функции системы работают в соответствии с заданными требованиями и способны надёжно обрабатывать различные сценарии использования. В следующей таблице представлен свод тест-кейсов, использованных при тестировании системы.

Таблица 2: Functional Test Cases for B2B and B2B2C Flow

Test ID	Test Name	Description	Input	Expected Result	Status
TC01	Get B2B Access Token	Verify token generation using client credentials	Valid client_id & client_secret	HTTP 200 OK, access token (JWT) returned with correct scope	Pass
TC02	Authorize User	Verify authorization endpoint returns code after consent request	Valid authorization request	Authorization code returned, HTTP 302 redirect	Pass
TC03	User Login	Verify user authentication with valid credentials	Valid username & password	HTTP 200 OK, user authenticated successfully	Pass
TC04	User Consent	Verify user consent is recorded	User approves access	Consent stored, authorization continues	Pass
TC05	Get B2B2C Token	Exchange authorization code for access token	Valid auth code + client data	HTTP 200 OK, access token returned	Pass
TC06	Access Resource (B2B)	Verify access to protected resource using B2B token	Valid B2B access token	HTTP 200 OK, resource data returned	Pass
TC07	Access Resource (B2B2C)	Verify access using B2B2C token	Valid B2B2C token	HTTP 200 OK, resource data returned	Pass

Помимо позитивного тестирования, система также была протестирована с использованием негативных тестовых сценариев с целью проверки её способности корректно обрабатывать ошибочные ситуации. Данные тесты включают сценарии с некорректными учетными данными, истекшими токенами, а также попытками несанкционированного доступа. Ожидается, что система будет отклонять некорректные запросы и возвращать ответы, соответствующие стандарту BI SNAP, включая корректные HTTP-коды состояния и структурированные сообщения об ошибках. Таким образом, система демонстрирует не только корректную

работу в нормальных условиях, но и надежность и соответствие требованиям при возникновении аномальных ситуаций. В следующей таблице представлен обзор негативных тестовых случаев, использованных при тестировании системы.

Таблица 3: Negative Test Cases for B2B and B2B2C Flow

Test ID	Test Name	Description	Input	Expected Result	Status
TC08	Invalid Client Credentials	Verify system rejects invalid client credentials	Wrong client_id / client_secret	HTTP 401 Unauthorized, error: invalid_client	Pass
TC09	Invalid Authorization Code	Verify system rejects invalid or fake authorization code	Invalid or modified auth code	HTTP 400 Bad Request, error: invalid_grant	Pass
TC10	Expired Access Token	Verify system rejects expired token	Expired JWT token	HTTP 401 Unauthorized, error: token_expired	Pass
TC11	Invalid User Login	Verify login fails with incorrect password	Valid username, wrong password	HTTP 401 Unauthorized, login failed message	Pass
TC12	User Not Found	Verify system handles non-existent user	Invalid username	HTTP 404 Not Found or 401 Unauthorized	Pass
TC13	User Denies Consent	Verify flow stops when user denies consent	User denies access	Authorization denied, no authorization code issued	Pass
TC14	Missing Authorization Header	Verify access without token is rejected	No Authorization header	HTTP 403 Forbidden, access denied	Pass
TC15	Invalid Token Signature	Verify system rejects tampered token	Modified JWT token	HTTP 401 Unauthorized, error: invalid_token	Pass
TC16	Missing Required Parameters	Verify system handles incomplete request	Missing client_id or scope	HTTP 400 Bad Request, error: invalid_request	Pass

5.1.2. Примеры API-запросов и ответов

Тестирование также проводится путём анализа примеров запросов и ответов для каждого основного endpoint с целью проверки соответствия формата и структуры данных установленным спецификациям. Представленные примеры включают процессы получения access token, авторизации пользователя, а также доступа к ресурсному серверу с

использованием токенов B2B и B2B2C в соответствии с протоколом OAuth 2.0. Каждый ответ, генерируемый системой, оценивается с точки зрения корректности статус-кода, структуры payload и достоверности возвращаемой информации. Это направлено на обеспечение согласованности взаимодействия между системами и соответствия стандарту BI SNAP, установленному Bank Indonesia. Далее приведены примеры запросов и ответов для процессов получения B2B и B2B2C access token.

Заголовки запроса включают:

- X-CLIENT-KEY — идентификатор клиента;
- X-TIMESTAMP — временная метка запроса;
- X-SIGNATURE — цифровая подпись запроса.

Пример запроса на получение access token представлен в листинге 5.

```
1 POST /oauth/token HTTP/1.1
2 X-CLIENT-KEY: {{clientId}}
3 X-TIMESTAMP: {{timestamp}}
4 X-SIGNATURE: {{signature}}
5
6 {
7   "grantType": "client_credentials",
8   "additionalInfo": {
9     "clientName": "My Client App"
10  }
11 }
```

Листинг 5: Пример запроса на получение B2B access token

Ответ сервера содержит маркер доступа и параметры его действия (листинг 6).

```
1 {
2   "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
3   "tokenType": "Bearer",
4   "expiresIn": "900",
5   "additionalInfo": {
6     "clientId": "My Client App"
7   },
8 }
```

```

8     "responseCode": "2007300",
9     "responseMessage": "successful"
10 }

```

Листинг 6: Пример ответа сервера (B2B access token)

Параметр `expiresIn` задаёт время жизни токена в секундах.

В сценарии B2B2C используется тип авторизации `authorization_code`, при котором требуется предварительно полученный код авторизации.

Пример запроса приведён в листинге 7.

```

1 POST /oauth/token HTTP/1.1
2 X-CLIENT-KEY: {{clientId}}
3 X-TIMESTAMP: {{timestamp}}
4 X-SIGNATURE: {{signature}}
5
6 {
7     "grantType": "AUTHORIZATION_CODE",
8     "authCode": "49f308de-db89-4e90-ba93-4c10877ba5c6",
9     "additionalInfo": {
10         "clientName": "My Client App"
11     }
12 }

```

Листинг 7: Пример запроса на получение B2B2C access token

Ответ сервера содержит access token, а также информацию о сроках действия как access token, так и refresh token (листинг 8).

```

1 {
2     "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
3     "tokenType": "Bearer",
4     "accessTokenExpiryTime": "2026-05-04T08:49:10.44930862Z",
5     "refreshToken": "",
6     "refreshTokenExpiryTime": "2026-05-05T08:34:10.44930862Z",
7     "additionalInfo": {
8         "clientName": "My Client App"
9     },
10     "responseCode": "2007400",
11     "responseMessage": "successful"
12 }

```

5.1.3. Результаты

На основании результатов проведённого функционального тестирования установлено, что все основные сценарии системы выполняются в соответствии с ожидаемыми результатами. Процессы аутентификации и авторизации, включая получение B2B и B2B2C access token, а также доступ к ресурсному серверу, были успешно реализованы без существенных проблем.

В целом результаты тестирования показывают, что система соответствует функциональным требованиям, определённым стандартом VI SNAP. Таким образом, можно сделать вывод, что система корректно реализует механизм OAuth 2.0 и может использоваться в соответствии с установленными требованиями.

5.2. Тестирование производительности

В данном разделе представлена оценка производительности предложенной системы при различных уровнях нагрузки. Целью исследования является измерение ключевых показателей производительности, включая задержку (latency) и пропускную способность (throughput), с целью оценки эффективности, масштабируемости и отзывчивости системы[29]. Эффективность характеризует степень рационального использования вычислительных ресурсов при обработке запросов. Масштабируемость отражает способность системы сохранять приемлемый уровень производительности при увеличении нагрузки. Отзывчивость определяет скорость реакции системы на запросы пользователей и, как правило, измеряется с использованием показателя задержки.

5.2.1. Конфигурация экспериментальной среды

Оценка производительности проводилась в контролируемой среде с целью обеспечения согласованности и достоверности полученных результатов. Эксперименты выполнялись на системе под управлением Ubuntu с версией ядра Linux 5.15.0-122-generic (64-битная архитектура). Аппаратная конфигурация включала процессор Intel® Core™ i7-8565U с базовой тактовой частотой 1,80 ГГц, имеющий 4 ядра и 8 потоков, а также 15 ГБ оперативной памяти и 8,3 ГБ подкачки. Все испытания проводились в локальной среде, при этом во время тестирования отсутствовали другие ресурсоёмкие приложения, что позволило минимизировать влияние внешних факторов на результаты измерений. Тестирование производительности осуществлялось с использованием Apache JMeter — инструмента с открытым исходным кодом, предназначенного для нагрузочного тестирования и оценки производительности веб-приложений и API [30]. Оцениваемая система тестировалась посредством HTTP-запросов, имитирующих реальные сценарии использования. Аутентификация реализована с применением протокола OAuth 2.0, при котором токены доступа динамически запрашивались у сервера авторизации и передавались в заголовке Authorization каждого запроса, что обеспечивало безопасное и реалистичное взаимодействие с API. Нагрузочная модель была разработана для имитации одновременной активности пользователей при различных уровнях нагрузки, при этом выполнялось несколько сценариев тестирования с различным количеством виртуальных пользователей. Каждый тест проводился в течение фиксированного интервала времени, в ходе которого регистрировались ключевые показатели производительности, такие как задержка (latency) и пропускная способность (throughput), с целью анализа поведения системы, её стабильности и эффективности при возрастании нагрузки [31].

5.2.2. Результаты измерений (задержка и пропускная способность)

В данном разделе представлены результаты оценки производительности системы на основе ключевых метрик, а именно задержки (latency) и пропускной способности (throughput). Тестирование проводилось на нескольких API-эндпоинтах, представляющих сквозной (end-to-end) рабочий процесс системы, при этом полученные результаты отражают агрегированную производительность системы в целом.

Таблица 4: Performance Metrics for B2B API Endpoints

API Endpoint	Avg Latency (ms)	Throughput (req/sec)	Error %
Access Token B2B	26	166.62	0%
Access Resource Server	301	166.72	0%

Таблица 5: Performance Metrics for B2B2C API Endpoints

API Endpoint	Avg Latency (ms)	Throughput (req/sec)	Error %
Authorization	51167	18.82	0%
Login	59276	18.82	0%
Consent	56790	18.82	0%
Token Request	33810	18.82	0%
Refresh Token	25406	18.82	0%
Access Resource Server	3958	18.82	0%

5.2.3. Анализ полученных результатов

В данном разделе представлен анализ производительности системы на основе двух сценариев тестирования: сценария B2B и сквозного (end-to-

end) сценария B2B2C. Оценка проводится на основе ключевых метрик, таких как задержка (latency), пропускная способность (throughput) и уровень ошибок.

В сценарии B2B система демонстрирует высокую производительность с относительно низкой задержкой. Эндпоинт Access Token B2B имеет среднее время отклика 26 мс, тогда как для Access Resource Server оно составляет 301 мс, что находится в пределах допустимых значений. Кроме того, пропускная способность достигает примерно 166 запросов в секунду, что свидетельствует об эффективной обработке нагрузки. Уровень ошибок равен 0%, что подтверждает стабильную работу системы в процессе тестирования.

В отличие от этого, в сценарии B2B2C наблюдается значительное снижение производительности. Большинство эндпоинтов, таких как Authorization, Login и Consent, демонстрируют крайне высокую задержку, превышающую 50 000 мс (около 50 секунд), что указывает на существенные задержки в процессе аутентификации. Несмотря на то, что эндпоинт Access Resource Server показывает более низкую задержку (3 958 мс), общая производительность системы остаётся ограниченной. Пропускная способность также значительно снижается до примерно 18,8 запросов в секунду, что указывает на наличие узкого места (bottleneck) в системе. При этом уровень ошибок остаётся равным 0%, что свидетельствует о стабильности системы, несмотря на снижение производительности.

В целом результаты показывают, что сложность процесса аутентификации в сценарии B2B2C оказывает существенное влияние на увеличение задержки и снижение пропускной способности. В то же время более простой сценарий B2B демонстрирует более высокую эффективность и масштабируемость. Следовательно, оптимизация должна быть сосредоточена на процессах аутентификации для повышения общей производительности системы.

Заключение

В ходе выполнения данной работы была разработана и реализована система авторизации на основе протокола OAuth 2.0, соответствующая стандарту BI-SNAP, с использованием архитектуры микросервисов. Разработанное решение поддерживает сценарии взаимодействия B2B и B2B2C и включает API-шлюз, обеспечивающий централизованную и безопасную обработку запросов.

В результате были получены следующие результаты:

1. Разработана и реализована система авторизации на основе протокола OAuth 2.0, поддерживающая сценарии B2B и B2B2C в соответствии со стандартом BI-SNAP.
2. Разработан API-шлюз как централизованная точка входа для клиентских запросов, реализующий механизмы OAuth 2.0 в соответствии со стандартом BI-SNAP, выполняющий валидацию запросов и обеспечивающий безопасное взаимодействие между клиентскими приложениями и сервером ресурсов.
3. Определена и реализована структура системы, включающая основные компоненты: Authorization, Login, Consent, Token Request, Refresh Token и Access Resource Server, обеспечивающие полный цикл аутентификации и авторизации.
4. Проведено экспериментальное исследование разработанной системы:
 - в сценарии B2B достигнуты низкая латентность, высокий throughput и отсутствие ошибок;
 - в сценарии B2B2C обеспечена стабильность функционирования системы без ошибок, однако зафиксировано увеличение латентности и снижение throughput, обусловленные повышенной сложностью процессов аутентификации.

5. Установлено, что использование архитектуры микросервисов является целесообразным для построения масштабируемой и надежной системы авторизации, соответствующей требованиям стандарта BI-SNAP.
6. Разработанная система ориентирована на интеграцию с ядром электронных денежных средств (e-money core system) и обеспечивает стандартизированный и безопасный уровень авторизации для поддержки дальнейшей интероперабельности.

Список литературы

- [1] Wikipedia. History of the Internet. — URL: https://en.wikipedia.org/wiki/History_of_the_Internet (дата обращения: 20 июня 2025 г.).
- [2] A Brief History of the Internet / Barry M. Leiner, Vinton G. Cerf, David D. Clark et al. // [ACM SIGCOMM Computer Communication Review](https://doi.org/10.1145/1629607.1629613). — 2009. — Vol. 39, no. 5. — P. 22–31. — URL: <https://doi.org/10.1145/1629607.1629613>.
- [3] Evans Peter C., Basole Rahul C. Revealing the API ecosystem and enterprise strategy via visual analytics // *Communications of the ACM*. — 2016. — Vol. 59, no. 2. — P. 26–28.
- [4] Foundation OWASP. Broken Access Control. — URL: https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/ (дата обращения: 21 апреля 2026 г.).
- [5] Microsoft. What Is SAML (Security Assertion Markup Language)? — URL: <https://www.microsoft.com/en-us/security/business/security-101/what-is-security-assertion-markup-language-saml> (дата обращения: 1 июля 2025 г.).
- [6] Google. Why and when to use API keys. — URL: <https://cloud.google.com/endpoints/docs/openapi/when-why-api-key> (дата обращения: 9 июля 2025 г.).
- [7] Jones Michael, Bradley John, Sakimura Nat. JSON Web Token (JWT). — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 21 апреля 2026 г.).
- [8] jwt.io. Introduction to JSON Web Tokens. — URL: <https://jwt.io/introduction#what-is-json-web-token> (дата обращения: 20 июля 2025 г.).

- [9] Newman Sam. Building Microservices. — O'Reilly Media, 2015.
- [10] Hardt Dick. The OAuth 2.0 Authorization Framework. — URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 2 июня 2025 г.).
- [11] Team Spring. Spring Authorization Server Reference Documentation. — URL: <https://docs.spring.io/spring-authorization-server/reference/> (дата обращения: 22 апреля 2026 г.).
- [12] Team Spring. Spring Security Reference Documentation. — URL: <https://docs.spring.io/spring-security/reference/> (дата обращения: 22 апреля 2026 г.).
- [13] Team Spring. Spring Data JPA Reference Documentation. — URL: <https://docs.spring.io/spring-data/jpa/reference/> (дата обращения: 22 апреля 2026 г.).
- [14] Team Spring. Spring Cloud Gateway Reference Documentation. — URL: <https://docs.spring.io/spring-cloud-gateway/reference/> (дата обращения: 22 апреля 2026 г.).
- [15] Team Spring. Spring WebFlux Reference Documentation. — URL: <https://docs.spring.io/spring-framework/reference/web/webflux.html> (дата обращения: 22 апреля 2026 г.).
- [16] Community R2DBC. R2DBC Specification. — URL: <https://r2dbc.io/> (дата обращения: 22 апреля 2026 г.).
- [17] teams Bank Indonesia. National Open API Payment Standard (SNAP). — URL: <https://www.bi.go.id/en/layanan/standar/snap/default.aspx> (дата обращения: 6 августа 2025 г.).
- [18] NIST. Digital Signature Standard (DSS), FIPS PUB 186-4. — URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf> (дата обращения: 20 апреля 2026 г.).

- [19] Rivest Ronald L., Shamir Adi, Adleman Leonard. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. — URL: <https://people.csail.mit.edu/rivest/Rsapaper.pdf> (дата обращения: 20 апреля 2026 г.).
- [20] NIST. Secure Hash Standard (SHS), FIPS PUB 180-4. — URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf> (дата обращения: 20 апреля 2026 г.).
- [21] Moriarty Kathleen et al. PKCS #1: RSA Cryptography Specifications Version 2.2. — URL: <https://datatracker.ietf.org/doc/html/rfc8017> (дата обращения: 20 апреля 2026 г.).
- [22] Schneier Bruce. Applied Cryptography: Protocols, Algorithms, and Source Code in C. — John Wiley & Sons, 1996.
- [23] Krawczyk Hugo, Bellare Mihir, Canetti Ran. HMAC: Keyed-Hashing for Message Authentication. — URL: <https://datatracker.ietf.org/doc/html/rfc2104> (дата обращения: 24 апреля 2026 г.).
- [24] NIST. The Keyed-Hash Message Authentication Code (HMAC), FIPS PUB 198-1. — URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.198-1.pdf> (дата обращения: 24 апреля 2026 г.).
- [25] Kelly Sean, Frankel Sheila. Using HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512 with IPsec. — URL: <https://datatracker.ietf.org/doc/html/rfc4868> (дата обращения: 24 апреля 2026 г.).
- [26] Bray Tim. The JavaScript Object Notation (JSON) Data Interchange Format. — URL: <https://datatracker.ietf.org/doc/html/rfc8259> (дата обращения: 24 апреля 2026 г.).
- [27] NIST. Recommendation for Applications Using Approved Hash Algorithms (SP 800-107 Rev.1). — URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-107r1.pdf> (дата обращения: 24 апреля 2026 г.).

- [28] Fielding Roy T., Reschke Julian. Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content. — URL: <https://datatracker.ietf.org/doc/html/rfc7231> (дата обращения: 24 апреля 2026 г.).
- [29] Barber Scott. Web Performance Testing for Dummies. — Wiley, 2011.
- [30] Foundation Apache Software. Apache JMeter User Manual. — URL: <https://jmeter.apache.org/> (дата обращения: 26 марта 2026 г.).
- [31] Gregg Brendan. Systems Performance: Enterprise and the Cloud. — Pearson, 2020.